

Chapter 2 — Hashing / Hash Map / Hash Set

0. What You Will Learn in This Chapter

Hashing is one of the most important coding interview patterns after basic array traversal.

Basic traversal lets you remember a few variables:

```
sum
count
maxVal
minVal
best
```

But many interview problems require remembering **many values** and checking them quickly.

Hashing helps you answer questions like:

- Have I seen this value before?
- How many times has this value appeared?
- What index did this value appear at?
- Does the complement exist?
- Are two strings made of the same character frequencies?
- Can I group items by a common signature?
- How many previous prefix sums match what I need?
- Which elements are common between two arrays?

By the end of this chapter, you should be comfortable with:

- unordered_map
- unordered_set
- frequency counting
- duplicate detection
- complement lookup
- value-to-index mapping
- grouping by key
- character frequency arrays
- prefix sum + hash map
- hash set expansion
- common edge cases
- common C++ hash map mistakes

Core LeetCode problems covered in this chapter:

- 1.1. Contains Duplicate
- 2.2. Two Sum
- 3.3. Valid Anagram
- 4.4. Group Anagrams
- 5.5. Longest Consecutive Sequence
- 6.6. Subarray Sum Equals K
- 7.7. First Unique Character in a String
- 8.8. Ransom Note
- 9.9. Intersection of Two Arrays

10.10.	Intersection of Two Arrays II
11.11.	Isomorphic Strings
12.12.	Valid Sudoku
13.13.	4Sum II

1. What Is Hashing?

Hashing is a technique that lets you store and retrieve data quickly using a key. In C++, the two most common hashing containers are:

```
unordered_map
unordered_set
```

1.1 Hash Set

A hash set stores only keys.

Use it when you only care whether something exists.

Example:

```
unordered_set<int> seen;

seen.insert(5);
seen.insert(10);

if (seen.count(5)) {
    cout << "5 exists";
}
```

A hash set answers:

Have I seen this value before?

1.2 Hash Map

A hash map stores key-value pairs.

Example:

```
unordered_map<int, int> freq;

freq[10] = 3;
freq[20] = 5;
```

Here:

- key: 10
- value: 3

A hash map answers:

What information is associated with this key?

Common uses:

number	frequency

Key	Value
number	index
character	count
prefix sum	number of times seen
string signature	group of strings
value	next greater value
node	cloned node

2. Why Hashing Exists

Many brute force problems repeatedly search for values.

Example:

Given:

```
nums = {2, 7, 11, 15}
target = 9
```

Find two numbers that add to target.

Brute force:

```
for (int i = 0; i < nums.size(); i++) {
    for (int j = i + 1; j < nums.size(); j++) {
        if (nums[i] + nums[j] == target) {
            return {i, j};
        }
    }
}
```

Time: $O(n^2)$

Why?

For every number, we search through the rest of the array.

Hashing improves this.

For each number x , we ask:

```
need = target - x
```

Then check quickly:

```
if (need exists in map)
```

This gives average $O(1)$ lookup.

So hashing exists to avoid repeated searching.

3. Core Need / Why Use Hashing Over Other Patterns?

Hashing is useful when you need **memory of many previous values**.

Basic traversal can remember one or two things:

```
minPrice
maxVal
sum
count
```

But what if you need to remember every value seen so far?

Example:

```
nums = {4, 1, 9, 2, 7}
```

If the current number is x , and you need to know whether target - x appeared earlier, a single variable is not enough.

You need a collection of seen values:

```
unordered_set<int> seen;
```

Or a map from value to index:

```
unordered_map<int, int> index;
```

3.1 Why Not Basic Traversal?

Basic traversal cannot quickly answer:

- Did I see 17 before?
- How many times did x appear?
- What index did x appear at?
- How many prefix sums equal prefix - k ?

Hashing can.

3.2 Why Not Sorting?

Sorting can solve some duplicate and frequency problems, but it changes order and costs $O(n \log n)$.

Example:

Contains Duplicate can be solved by sorting:

```
sort(nums.begin(), nums.end());

for (int i = 1; i < nums.size(); i++) {
    if (nums[i] == nums[i - 1]) {
        return true;
    }
}
```

Time: $O(n \log n)$

Hashing solution:

```
unordered_set<int> seen;
```

Time: $O(n)$ average.

Sorting is not always bad, but hashing is usually better when you need fast lookup and do not need sorted order.

3.3 Why Not Two Pointers?

Two pointers work well when the array is sorted.

For unsorted Two Sum:

```
nums = {3, 2, 4}
```

```
target = 6
```

Two pointers cannot safely move unless array is sorted.

Hashing is better because it does not require sorting.

3.4 Why Not Binary Search?

Binary search needs sorted data.

If data is unsorted and original order or indices matter, hashing is usually better.

4. When to Use Hashing

Use hashing when the problem has one of these signals.

4.1 Fast Lookup

Problem says:

- “check if exists”
- “contains”
- “seen before”
- “find pair”
- “duplicate”
- “intersection”

Use:

```
unordered_set
```

Examples:

- Contains Duplicate
 - Two Sum
 - Intersection of Two Arrays
 - Happy Number
-

4.2 Frequency Counting

Problem says:

- “frequency”
- “count occurrences”
- “anagram”
- “same characters”
- “most common”

- “appears how many times”

Use:

```
unordered_map<int, int> freq;
unordered_map<char, int> freq;
```

or for lowercase English letters:

```
vector<int> freq(26, 0);
```

Examples:

- Valid Anagram
 - Ransom Note
 - First Unique Character
 - Sort Characters By Frequency
 - Top K Frequent Elements
-

4.3 Value to Index Mapping

Problem says:

- “return indices”
- “previous index”
- “distance between same values”
- “last seen index”

Use:

```
unordered_map<int, int> index;
```

Examples:

- Two Sum
 - Contains Duplicate II
 - Isomorphic Strings
-

4.4 Grouping by Signature

Problem says:

- “group anagrams”
- “group by same property”
- “same pattern”
- “same transformed form”

Use:

```
unordered_map<string, vector<string>> groups;
```

Examples:

- Group Anagrams
 - Group Shifted Strings
-

4.5 Prefix Sum + Hash Map

Problem says:

- “count subarrays with sum k”
- “subarray sum equals”
- “continuous subarray”
- “number of subarrays”
- “same prefix remainder”

Use:

```
unordered_map<int, int> prefixCount;
```

Examples:

- Subarray Sum Equals K
- Subarray Sums Divisible by K
- Continuous Subarray Sum
- Contiguous Array

4.6 Unique Tracking

Problem asks:

- “all values unique”
- “unique occurrences”
- “remove duplicates”
- “detect repeated values”

Use:

```
unordered_set<int> seen;
```

Examples:

- Unique Number of Occurrences
- Contains Duplicate
- Intersection of Two Arrays

5. When NOT to Use Hashing

Hashing is powerful, but not always best.

5.1 Need Sorted Order

`unordered_map` and `unordered_set` do not keep keys sorted.

If you need sorted keys, use:

```
map
set
multiset
```

Examples:

- My Calendar
- Sliding Window Median
- Contains Duplicate III

5.2 Input Is Sorted and $O(1)$ Space Is Desired

For sorted Two Sum, use two pointers instead of hash map.

Example:

```
numbers = {2, 7, 11, 15}
target = 9
```

Two pointers gives:

- $O(n)$ time
- $O(1)$ space

Hashing gives:

- $O(n)$ time
 - $O(n)$ space
-

5.3 Need Top K

Hashing can count frequencies, but to get top K you often need:

- heap
- bucket sort
- sorting

Example:

Top K Frequent Elements:

2. Count with hash map
3. Use heap/bucket/sort

Hashing is only part of the solution.

5.4 Memory Is Tight

Hash tables use extra space.

Sometimes sorting is acceptable if extra space must be minimized.

5.5 Need Range Min/Max Queries

Hash maps do not answer range queries like:

- minimum in range
- maximum in range
- dynamic sum range

Use:

- prefix sum
- segment tree
- Fenwick tree
- monotonic deque

depending on problem.

6. Hash Map vs Hash Set

6.1 Use `unordered_set` When You Only Need Existence

Example:

```
unordered_set<int> seen;
```

```
if (seen.count(x)) {  
    return true;  
}
```

```
seen.insert(x);
```

Questions answered:

- Does x exist?
- Have I seen x before?

- Is x unique?
-

6.2 Use `unordered_map` When You Need Associated Information

Example:

```
unordered_map<int, int> freq;
freq[x]++;
```

Questions answered:

- How many times did x appear?
 - What index was x seen at?
 - What group does this key map to?
 - How many times has this prefix appeared?
-

7. C++ Tools Needed

7.1 `unordered_set`

```
unordered_set<int> st;
```

Operations:

```
st.insert(x);
st.count(x);
st.find(x);
st.erase(x);
st.size();
st.empty();
```

Example:

```
if (st.count(x)) {
    // x exists
}
```

7.2 `unordered_map`

```
unordered_map<int, int> mp;
```

Operations:

```
mp[x] = 5;
mp[x]++;
mp.count(x);
mp.find(x);
mp.erase(x);
mp.size();
```

7.3 Checking Existence Safely

Preferred:

```
if (mp.count(x)) {  
    // key exists  
}
```

or:

```
if (mp.find(x) != mp.end()) {  
    // key exists  
}
```

7.4 Important Warning About mp[x]

This line:

```
mp[x]
```

creates key x if it does not exist.

Example:

```
unordered_map<int, int> mp;  
  
cout << mp[100];
```

This prints 0, but it also inserts key 100.

So do not use mp[x] just to check if a key exists.

Wrong:

```
if (mp[x]) {  
    // maybe exists?  
}
```

Correct:

```
if (mp.count(x)) {  
    // exists  
}
```

7.5 Frequency Array for Characters

For lowercase letters:

```
vector<int> freq(26, 0);  
  
for (char c : s) {  
    freq[c - 'a']++;  
}
```

For uppercase letters:

```
vector<int> freq(26, 0);
```

```
for (char c : s) {  
    freq[c - 'A']++;  
}
```

For ASCII:

```
vector<int> freq(128, 0);
```

```
for (char c : s) {  
    freq[c]++;  
}
```

7.6 Iterating Over Hash Map

```
for (auto p : mp) {  
    int key = p.first;  
    int value = p.second;  
}
```

Better to avoid copying for large values:

```
for (auto& p : mp) {  
    int key = p.first;  
    int value = p.second;  
}
```

7.7 Iteration Order Warning

`unordered_map` and `unordered_set` do not guarantee order.

This means:

```
for (auto p : mp)
```

does not follow insertion order or sorted order.

If order matters, do not rely on unordered containers.

8. Time and Space Complexity of Hashing

For `unordered_map` and `unordered_set`:

Average case:

insert	$O(1)$

Operation	Average Time
find/count	$O(1)$
erase	$O(1)$

Worst case:

$O(n)$, due to collisions.

In interviews, usually say:

Average $O(1)$, worst-case $O(n)$.

Space:

$O(n)$, because we may store many values.

9. Core Templates

Template 1 — Duplicate Detection

```
bool hasDuplicate(vector<int>& nums) {
    unordered_set<int> seen;

    for (int x : nums) {
        if (seen.count(x)) {
            return true;
        }

        seen.insert(x);
    }

    return false;
}
```

Use when:

- value should be unique
- need seen-before check

Template 2 — Frequency Count

```
unordered_map<int, int> freq;

for (int x : nums) {
    freq[x]++;
}
```

Then:

```
for (auto& p : freq) {
    int value = p.first;
```

```
int count = p.second;
}
```

Use when:

- count occurrences
- find duplicates
- compare frequencies
- most common element

Template 3 — Character Frequency

```
vector<int> freq(26, 0);
```

```
for (char c : s) {
    freq[c - 'a']++;
}
```

Use when:

- lowercase English letters only
- anagram-like problems
- ransom note
- first unique character

Template 4 — Value to Index

```
unordered_map<int, int> index;
```

```
for (int i = 0; i < nums.size(); i++) {
    index[nums[i]] = i;
}
```

Use when:

- need location of a value
- return indices
- compare index distance

Template 5 — One-Pass Two Sum

```
unordered_map<int, int> index;
```

```
for (int i = 0; i < nums.size(); i++) {
    int need = target - nums[i];

    if (index.count(need)) {
        return {index[need], i};
    }
}
```

```
    index[nums[i]] = i;
}
```

Important:

Check complement before inserting current value.

Template 6 — Group by Signature

```
unordered_map<string, vector<string>> groups;

for (string word : words) {
    string key = word;
    sort(key.begin(), key.end());

    groups[key].push_back(word);
}

vector<vector<string>> result;

for (auto& p : groups) {
    result.push_back(p.second);
}

return result;
```

Use when:

- same transformed key means same group

Template 7 — Prefix Sum + Hash Map Count

```
unordered_map<int, int> count;
count[0] = 1;

int prefix = 0;
int answer = 0;

for (int x : nums) {
    prefix += x;

    int need = prefix - k;

    if (count.count(need)) {
        answer += count[need];
    }
}
```

```
    count[prefix]++;  
}
```

```
return answer;
```

Use when:

- count subarrays with exact sum k

Template 8 — Hash Set Expansion

```
unordered_set<int> st(nums.begin(), nums.end());
```

```
int best = 0;
```

```
for (int x : st) {  
    if (!st.count(x - 1)) {  
        int current = x;  
        int length = 1;  
  
        while (st.count(current + 1)) {  
            current++;  
            length++;  
        }  
  
        best = max(best, length);  
    }  
}
```

```
return best;
```

Use for:

- Longest Consecutive Sequence

Template 9 — Two Frequency Maps Comparison

```
unordered_map<char, int> a;  
unordered_map<char, int> b;
```

```
for (char c : s) {  
    a[c]++;  
}
```

```
for (char c : t) {  
    b[c]++;  
}
```

```
return a == b;
```

Usually, for fixed alphabets, prefer arrays.

Template 10 — Unique Occurrence Check

```
unordered_map<int, int> freq;

for (int x : arr) {
    freq[x]++;
}

unordered_set<int> seenCounts;

for (auto& p : freq) {
    int count = p.second;

    if (seenCounts.count(count)) {
        return false;
    }

    seenCounts.insert(count);
}

return true;
```

10. Major Variations of Hashing

Existence check	unordered_set	Contains Duplicate
Frequency count	unordered_map	Valid Anagram
Value to index	unordered_map	Two Sum
Grouping	unordered_map<string, vector<...>>	Group Anagrams
Prefix sum count	unordered_map<int, int>	Subarray Sum Equals K
Set expansion	unordered_set	Longest Consecutive

Variation	Main Tool	Example
		Sequence
Pattern mapping	two maps / arrays	Isomorphic Strings
Board validation	sets / frequency checks	Valid Sudoku
Multi-array counting	hash map of pair sums	4Sum II

11. How to Identify Hashing in Interviews

Ask:

4. Do I need to check if a value appeared before?
5. Do I need fast lookup?
6. Do I need frequency counts?
7. Do I need to return indices?
8. Do I need to group items by a shared property?
9. Do I need to compare two collections by counts?
10. Do I need to count subarrays with some prefix relationship?
11. Would brute force repeatedly search the array?
12. Does sorted order not matter?
13. Is $O(n)$ extra space acceptable?

If yes, hashing is likely useful.

12. Full Problem Walkthrough 1 — LeetCode 217: Contains Duplicate

12.1 Problem Summary

Given an integer array `nums`, return `true` if any value appears at least twice.

Example:

```
nums = {1, 2, 3, 1}
```

Output:

```
true
```

Example:

```
nums = {1, 2, 3, 4}
```

Output:

```
false
```

12.2 How to Identify This Pattern

The problem asks:

- Does a value repeat?
- Have we seen this before?

This is a direct hash set problem.

12.3 Brute Force Idea

Compare every pair.

```
for (int i = 0; i < nums.size(); i++) {  
    for (int j = i + 1; j < nums.size(); j++) {  
        if (nums[i] == nums[j]) {  
            return true;  
        }  
    }  
}
```

Time: $O(n^2)$

12.4 Optimized Intuition

Scan once.

Maintain a set of seen values.

For each value:

- if already in set, duplicate exists
 - otherwise insert it
-

12.5 C++ Code

```
class Solution {  
public:  
    bool containsDuplicate(vector<int>& nums) {  
        unordered_set<int> seen;  
  
        for (int x : nums) {  
            if (seen.count(x)) {  
                return true;  
            }  
  
            seen.insert(x);  
        }  
  
        return false;  
    }  
};
```

12.6 Code Explanation

```
unordered_set<int> seen;
```

Stores numbers already visited.

```
if (seen.count(x))
```

Checks if x appeared before.

```
seen.insert(x);
```

Stores current number.

12.7 Dry Run

```
nums = {1, 2, 3, 1}
```

1	{}	insert 1
2	{1}	insert 2
3	{1,2}	insert 3
1	{1,2,3}	duplicate found

Return true.

12.8 Correctness Explanation

If a duplicate exists, the second occurrence will find the first occurrence already in the set. If no value is ever found in the set before insertion, all values are unique.

12.9 Complexity

Time: $O(n)$ average

Each lookup and insert is average $O(1)$.

Space: $O(n)$

In the worst case, all elements are unique and stored.

12.10 Edge Cases

- Empty array: false
- One element: false
- All same values: true
- Negative numbers: works
- Large values: works

12.11 Common Mistake

Wrong:

```
seen.insert(x);
```

```
if (seen.count(x)) {  
    return true;  
}
```

This always returns true after inserting first element.
Correct order:

```
if (seen.count(x)) {  
    return true;  
}  
  
seen.insert(x);
```

12.12 Similar Problems

- Contains Duplicate II
- Contains Duplicate III
- Intersection of Two Arrays
- Happy Number

13. Full Problem Walkthrough 2 — LeetCode 1: Two Sum

13.1 Problem Summary

Given an array `nums` and integer `target`, return indices of two numbers that add to `target`.

Example:

```
nums = {2, 7, 11, 15}  
target = 9
```

Output:

```
{0, 1}
```

13.2 How to Identify This Pattern

The problem asks:

- find pair
- return indices
- complement lookup

For each number `x`, need to check if:

```
target - x
```

has appeared before.

Use hash map:

```
value -> index
```

13.3 Brute Force Idea

Try every pair.

Time: $O(n^2)$

13.4 Optimized Intuition

Scan once.

For each index i:

need = target - nums[i]

If need was seen earlier, return earlier index and current index.

Otherwise store current number and index.

13.5 C++ Code

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> index;

        for (int i = 0; i < nums.size(); i++) {
            int need = target - nums[i];

            if (index.count(need)) {
                return {index[need], i};
            }

            index[nums[i]] = i;
        }

        return {};
    }
};
```

13.6 Code Explanation

unordered_map<int, int> index;

Stores value to index.

int need = target - nums[i];

Complement needed.

if (index.count(need))

If complement appeared before, answer found.

index[nums[i]] = i;

Store current value for future numbers.

13.7 Why Check Before Insert?

If we insert first, we may use the same element twice.

Example:

```
nums = {3}
target = 6
```

If you insert 3 and then check for 3, you may incorrectly pair index 0 with itself.

Correct:

14. check complement
 15. insert current
-

13.8 Dry Run

```
nums = {2, 7, 11, 15}
target = 9
```

0	2	7	{}	store 2 -> 0
1	7	2	{2:0}	found, return {0,1}

13.9 Complexity

Time: $O(n)$ average

Space: $O(n)$

13.10 Edge Cases

- Duplicate values: {3,3}, target 6
 - Negative numbers
 - Zeroes
 - Multiple answers
 - Large target values
-

13.11 Common Mistakes

Mistake 1: Using a set only

A set tells existence but not index.

Need map.

Mistake 2: Sorting

Sorting loses original indices unless you store them.

Hash map is cleaner.

13.12 Similar Problems

- Two Sum II
- 3Sum
- 4Sum

- 4Sum II
 - Two Sum IV
-

14. Full Problem Walkthrough 3 — LeetCode 242: Valid Anagram

14.1 Problem Summary

Given two strings *s* and *t*, return true if *t* is an anagram of *s*.

Example:

```
s = "anagram"
t = "nagaram"
```

Output:

```
true
```

14.2 How to Identify This Pattern

Anagram means:

- same characters
- same frequencies

This is frequency counting.

14.3 Brute Force Idea

For each character in *s*, search and remove it from *t*.

Time: $O(n^2)$

14.4 Optimized Intuition

Count characters in *s*.

Subtract characters in *t*.

All counts should become zero.

14.5 C++ Code — Lowercase English Letters

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) {
            return false;
        }

        vector<int> freq(26, 0);

        for (char c : s) {
            freq[c - 'a']++;
        }
    }
};
```

```

    }

    for (char c : t) {
        freq[c - 'a']--;
    }

    for (int count : freq) {
        if (count != 0) {
            return false;
        }
    }

    return true;
}
};

```

14.6 Code Explanation

```
vector<int> freq(26, 0);
```

Stores counts for 'a' to 'z'.

```
freq[c - 'a']++;
```

Maps character to index.

Example:

'a' - 'a' = 0

'b' - 'a' = 1

14.7 Dry Run

s = "aabb"

t = "baba"

After s:

a	2
b	2

After subtracting t, all counts become 0.

Return true.

14.8 Hash Map Version for General Characters

```

class Solution {
public:

```

```
bool isAnagram(string s, string t) {
    if (s.size() != t.size()) {
        return false;
    }

    unordered_map<char, int> freq;

    for (char c : s) {
        freq[c]++;
    }

    for (char c : t) {
        freq[c]--;
    }

    for (auto& p : freq) {
        if (p.second != 0) {
            return false;
        }
    }

    return true;
}
};
```

14.9 Complexity

For lowercase English letters:

Time: $O(n)$

Space: $O(1)$, because 26 is constant.

For general characters:

Time: $O(n)$

Space: $O(k)$, where k is number of distinct characters.

14.10 Edge Cases

- Different lengths
 - Empty strings
 - Same letters, different counts
 - Uppercase/lowercase distinction
 - Non-English characters, if applicable
-

14.11 Common Mistake

Using sets:

```
unordered_set<char> a;  
unordered_set<char> b;
```

Fails for:

```
s = "aab"  
t = "abb"
```

Both have {a,b}, but counts differ.

14.12 Similar Problems

- Group Anagrams
- Find All Anagrams in a String
- Permutation in String
- Ransom Note

15. Full Problem Walkthrough 4 — LeetCode 383: Ransom Note

15.1 Problem Summary

Given strings ransomNote and magazine, return true if ransomNote can be constructed from letters in magazine.

Each magazine letter can be used once.

Example:

```
ransomNote = "aa"  
magazine = "aab"
```

Output:

```
true
```

15.2 How to Identify This Pattern

This asks:

- do we have enough characters?
- each character has limited frequency

Use frequency counting.

15.3 C++ Code

```
class Solution {  
public:  
    bool canConstruct(string ransomNote, string magazine) {  
        vector<int> freq(26, 0);  
  
        for (char c : magazine) {  
            freq[c - 'a']++;  
        }  
    }  
};
```

```

    }

    for (char c : ransomNote) {
        freq[c - 'a']--;

        if (freq[c - 'a'] < 0) {
            return false;
        }
    }

    return true;
}
};

```

15.4 Code Explanation

First count all available letters from magazine.

Then spend letters for ransomNote.

If any count becomes negative, we needed more of that letter than available.

15.5 Dry Run

```

ransomNote = "aa"
magazine = "aab"

```

Magazine count:

a	2
b	1

Use ransomNote:

- use a: count becomes 1
- use a: count becomes 0

Return true.

15.6 Complexity

Time: $O(n + m)$

Space: $O(1)$

15.7 Edge Cases

- Empty ransom note: true
- Magazine shorter than ransom note: usually false, but still code handles
- Repeated letters
- Exact match

15.8 Common Mistakes

Mistake: Only checking existence

Need enough copies, not just presence.

Example:

```
ransomNote = "aa"  
magazine = "ab"
```

a exists, but not enough times.

15.9 Similar Problems

- Valid Anagram
 - First Unique Character
 - Find All Anagrams in a String
-

16. Full Problem Walkthrough 5 — LeetCode 49: Group Anagrams

16.1 Problem Summary

Given an array of strings, group anagrams together.

Example:

```
strs = {"eat", "tea", "tan", "ate", "nat", "bat"}
```

Output:

```
{  
  {"eat", "tea", "ate"},  
  {"tan", "nat"},  
  {"bat"}  
}
```

Order does not usually matter.

16.2 How to Identify This Pattern

Problem says:

- group strings
- anagrams
- same character frequency

Use hash map:

```
signature -> list of words
```

16.3 Brute Force Idea

Compare every string with every other string using anagram check.

Time can be $O(n^2 * m)$

16.4 Optimized Intuition

All anagrams become equal after sorting characters.

Example:

```
"eat" -> "aet"  
"tea" -> "aet"  
"ate" -> "aet"
```

Use sorted string as key.

16.5 C++ Code — Sorting Key

```
class Solution {  
public:  
    vector<vector<string>> groupAnagrams(vector<string>& strs) {  
        unordered_map<string, vector<string>> groups;  
  
        for (string word : strs) {  
            string key = word;  
            sort(key.begin(), key.end());  
  
            groups[key].push_back(word);  
        }  
  
        vector<vector<string>> result;  
  
        for (auto& p : groups) {  
            result.push_back(p.second);  
        }  
  
        return result;  
    }  
};
```

16.6 Dry Run

```
strs = {"eat", "tea", "tan", "ate", "nat", "bat"}
```

eat	aet	aet: eat
tea	aet	aet: eat, tea
tan	ant	ant: tan
ate	aet	aet: eat, tea, ate

word	sorted key	group
nat	ant	ant: tan, nat
bat	abt	abt: bat

16.7 Complexity

Let:

- n = number of strings
- m = average string length

Sorting each word costs $O(m \log m)$.

Total time: $O(n * m \log m)$

Space: $O(n * m)$

16.8 Alternative Code — Frequency Signature

This avoids sorting.

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> groups;

        for (string word : strs) {
            vector<int> freq(26, 0);

            for (char c : word) {
                freq[c - 'a']++;
            }

            string key;

            for (int count : freq) {
                key += '#';
                key += to_string(count);
            }

            groups[key].push_back(word);
        }

        vector<vector<string>> result;

        for (auto& p : groups) {
            result.push_back(p.second);
        }
    }
};
```

```
        return result;
    }
};
```

16.9 Why Use # in Key?

Without separators, counts can collide.

Example:

```
[1, 11]
[11, 1]
```

Both could look like:

```
111
```

With separator:

```
#1#11
#11#1
```

Different keys.

16.10 Edge Cases

- Empty string
- Single word
- All anagrams
- No anagrams
- Repeated characters

16.11 Common Mistakes

Mistake 1: Using original word as key

Wrong because "eat" and "tea" are different strings but same anagram group.

Mistake 2: Returning the map directly

Need convert map values into `vector<vector<string>>`.

16.12 Similar Problems

- Valid Anagram
- Find All Anagrams in a String
- Permutation in String
- Group Shifted Strings

17. Full Problem Walkthrough 6 — LeetCode 387: First Unique Character in a String

17.1 Problem Summary

Given a string *s*, return index of first non-repeating character.

If none exists, return -1.

Example:

```
s = "loveleetcode"
```

Output:

```
2
```

Because 'v' is first unique character.

17.2 How to Identify This Pattern

Need:

- count character frequencies
- preserve original order for answer

Use two passes:

16. count frequencies
17. scan original string for first count 1

17.3 C++ Code

```
class Solution {
public:
    int firstUniqChar(string s) {
        vector<int> freq(26, 0);

        for (char c : s) {
            freq[c - 'a']++;
        }

        for (int i = 0; i < s.size(); i++) {
            if (freq[s[i] - 'a'] == 1) {
                return i;
            }
        }

        return -1;
    }
};
```

17.4 Dry Run

```
s = "loveleetcode"
```

Frequencies:

l	2
o	2
v	1
e	4
t	1
c	1
d	1

Scan from left:

0	l	2	skip
1	o	2	skip
2	v	1	return 2

17.5 Complexity

Time: $O(n)$

Space: $O(1)$, fixed 26 letters.

17.6 Common Mistake

Iterating through unordered map and returning first count 1.

Wrong because unordered map does not preserve string order.

Correct:

Scan original string again.

17.7 Similar Problems

- Ransom Note
 - Valid Anagram
 - Sort Characters By Frequency
 - First Letter to Appear Twice
-

18. Full Problem Walkthrough 7 — LeetCode 349: Intersection of Two Arrays

18.1 Problem Summary

Return unique intersection of two arrays.

Example:

```
nums1 = {1, 2, 2, 1}
nums2 = {2, 2}
```

Output:

```
{2}
```

18.2 How to Identify This Pattern

Need:

- check membership in another array
- result should be unique

Use hash set.

18.3 C++ Code

```
class Solution {
public:
    vector<int> intersection(vector<int>& nums1, vector<int>& nums2) {
        unordered_set<int> set1(nums1.begin(), nums1.end());
        unordered_set<int> resultSet;

        for (int x : nums2) {
            if (set1.count(x)) {
                resultSet.insert(x);
            }
        }

        vector<int> result;

        for (int x : resultSet) {
            result.push_back(x);
        }

        return result;
    }
};
```

18.4 Code Explanation

```
unordered_set<int> set1(nums1.begin(), nums1.end());
```

Stores unique values from nums1.

```
resultSet.insert(x);
```

Ensures result has no duplicates.

18.5 Dry Run

nums1 = {1, 2, 2, 1}

nums2 = {2, 2}

set1 = {1,2}

Process nums2:

2	yes	{2}
2	yes	{2}

18.6 Complexity

Time: $O(n + m)$ average

Space: $O(n + m)$

18.7 Edge Cases

- No intersection
 - One array empty
 - Duplicates
 - Negative numbers
 - Output order does not matter
-

18.8 Similar Problems

- Intersection of Two Arrays II
 - Find the Difference of Two Arrays
 - Intersection of Multiple Arrays
-

19. Full Problem Walkthrough 8 — LeetCode 350: Intersection of Two Arrays II

19.1 Problem Summary

Return intersection of two arrays, including duplicates.

Example:

nums1 = {1, 2, 2, 1}

nums2 = {2, 2}

Output:

{2, 2}

19.2 How to Identify This Pattern

Unlike Intersection I, duplicates matter.
So use frequency map, not set.

19.3 C++ Code

```
class Solution {
public:
    vector<int> intersect(vector<int>& nums1, vector<int>& nums2) {
        unordered_map<int, int> freq;

        for (int x : nums1) {
            freq[x]++;
        }

        vector<int> result;

        for (int x : nums2) {
            if (freq[x] > 0) {
                result.push_back(x);
                freq[x]--;
            }
        }

        return result;
    }
};
```

19.4 Code Explanation

```
freq[x]++;
```

Counts available copies from nums1.

```
if (freq[x] > 0)
```

If copy is available, use it once.

```
freq[x]--;
```

Consume one copy.

19.5 Dry Run

```
nums1 = {1, 2, 2, 1}
```

```
nums2 = {2, 2}
```

Frequency from nums1:

1 -> 2
2 -> 2

Process nums2:

2	2	add 2	1
2	1	add 2	0

Result:

{2, 2}

19.6 Complexity

Time: $O(n + m)$ average

Space: $O(n)$

19.7 Common Mistake

Using set loses duplicates.

Intersection II requires counts.

19.8 Similar Problems

- Intersection of Two Arrays
- Find Common Characters
- Find the Difference of Two Arrays

20. Full Problem Walkthrough 9 — LeetCode 128: Longest Consecutive Sequence

20.1 Problem Summary

Given an unsorted array, return length of longest consecutive sequence.

Example:

nums = {100, 4, 200, 1, 3, 2}

Output:

4

Because sequence is:

1, 2, 3, 4

20.2 How to Identify This Pattern

Problem asks:

- consecutive numbers
- unsorted array
- $O(n)$ expected

Need fast existence lookup:

$x + 1$ exists?

$x - 1$ exists?

Use hash set.

20.3 Brute Force Idea

For each number, search if next number exists by scanning array.

Time: $O(n^2)$

Sorting gives $O(n \log n)$, but problem wants $O(n)$.

20.4 Optimized Intuition

Put all numbers in set.

Only start counting from sequence starts.

A number x starts a sequence if:

$x - 1$ does not exist

Then count:

$x, x+1, x+2, \dots$

20.5 C++ Code

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> st(nums.begin(), nums.end());

        int best = 0;

        for (int x : st) {
            if (!st.count(x - 1)) {
                int current = x;
                int length = 1;

                while (st.count(current + 1)) {
                    current++;
                    length++;
                }

                best = max(best, length);
            }
        }
    }
};
```

```

    }

    return best;
}
};

```

20.6 Dry Run

nums = {100, 4, 200, 1, 3, 2}

Set:

{100, 4, 200, 1, 3, 2}

100	no	yes	100
4	yes, 3	no	skip
200	no	yes	200
1	no	yes	1,2,3,4
3	yes, 2	no	skip
2	yes, 1	no	skip

Best = 4.

20.7 Why This Is $O(n)$

Each number is part of at most one sequence expansion.

We only expand from sequence starts.

Each number is visited a constant number of times on average.

Time: $O(n)$ average.

20.8 Edge Cases

- Empty array: return 0
- Duplicates
- Negative numbers
- One element
- Already consecutive
- No consecutive values

20.9 Common Mistakes

Mistake 1: Starting from every number

Can become $O(n^2)$.

Mistake 2: Sorting

Sorting works but violates $O(n)$ requirement.

Mistake 3: Not handling duplicates

Hash set removes duplicates naturally.

20.10 Similar Problems

- Longest Harmonious Subsequence
 - Missing Number
 - First Missing Positive, harder in-place variation
-

21. Full Problem Walkthrough 10 — LeetCode 560: Subarray Sum Equals K

21.1 Problem Summary

Given integer array `nums` and integer `k`, return number of continuous subarrays whose sum equals `k`.

Example:

```
nums = {1, 1, 1}
k = 2
```

Output:

```
2
```

Subarrays:

```
{1,1} at indices 0..1
{1,1} at indices 1..2
```

21.2 How to Identify This Pattern

Problem asks:

- count subarrays
- exact sum `k`
- array may contain negatives

Sliding window is not reliable with negative numbers.

Use prefix sum + hash map.

21.3 Brute Force Idea

Try every subarray.

Time: $O(n^2)$

21.4 Key Insight

Let prefix be sum from start to current index.

If a previous prefix is previous, then subarray sum is:

```
prefix - previous
```

We want:

prefix - previous = k

So:

previous = prefix - k

Therefore, at each index, count how many previous prefixes equal:

prefix - k

21.5 C++ Code

```
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<int, int> prefixCount;

        prefixCount[0] = 1;

        int prefix = 0;
        int answer = 0;

        for (int x : nums) {
            prefix += x;

            int need = prefix - k;

            if (prefixCount.count(need)) {
                answer += prefixCount[need];
            }

            prefixCount[prefix]++;
        }

        return answer;
    }
};
```

21.6 Code Explanation

prefixCount[0] = 1;

Represents empty prefix before array starts.
This handles subarrays starting at index 0.

prefix += x;

Current running sum.

```
need = prefix - k;
```

Previous prefix needed.

```
answer += prefixCount[need];
```

Every previous occurrence creates a valid subarray.

```
prefixCount[prefix]++;
```

Store current prefix for future subarrays.

21.7 Dry Run

```
nums = {1, 1, 1}
```

```
k = 2
```

Initial:

```
prefixCount = {0: 1}
```

1	1	-1	0	0
1	2	0	1	1
1	3	1	1	2

Return 2.

21.8 Dry Run With Negative Numbers

```
nums = {1, -1, 1}
```

```
k = 1
```

1	1	0	1	1
-1	0	-1	0	1
1	1	0	2	3

Valid subarrays:

```
{1}
```

```
{1,-1,1}
```

```
{1}
```

Answer: 3

21.9 Complexity

Time: $O(n)$ average

Space: $O(n)$

21.10 Edge Cases

- $k = 0$
 - negative numbers
 - subarray starts at index 0
 - repeated prefix sums
 - all zeroes
-

21.11 Common Mistakes

Mistake 1: Forgetting `prefixCount[0] = 1`

Misses subarrays starting at index 0.

Mistake 2: Using set instead of map

Need count of previous prefixes, not just existence.

Mistake 3: Updating current prefix count before checking

Correct order:

```
prefix += x;
answer += prefixCount[prefix - k];
prefixCount[prefix]++;
```

21.12 Similar Problems

- Subarray Sums Divisible by K
 - Continuous Subarray Sum
 - Binary Subarrays With Sum
 - Count Number of Nice Subarrays
 - Contiguous Array
-

22. Full Problem Walkthrough 11 — LeetCode 205: Isomorphic Strings

22.1 Problem Summary

Two strings are isomorphic if characters in one string can be replaced to get the other string.

Each character must map consistently.

Different characters cannot map to the same character.

Example:

```
s = "egg"
t = "add"
```

Output:

```
true
```

Mapping:

```
e -> a
g -> d
```

Example:

```
s = "foo"  
t = "bar"
```

Output:

```
false
```

22.2 How to Identify This Pattern

Problem asks:

- character mapping
- consistent relationship
- one-to-one mapping

Use hash maps or arrays.

Need both directions:

```
s -> t  
t -> s
```

22.3 C++ Code

```
class Solution {  
public:  
    bool isIsomorphic(string s, string t) {  
        vector<int> mapST(256, -1);  
        vector<int> mapTS(256, -1);  
  
        for (int i = 0; i < s.size(); i++) {  
            char a = s[i];  
            char b = t[i];  
  
            if (mapST[a] == -1 && mapTS[b] == -1) {  
                mapST[a] = b;  
                mapTS[b] = a;  
            } else {  
                if (mapST[a] != b || mapTS[b] != a) {  
                    return false;  
                }  
            }  
        }  
  
        return true;  
    }  
};
```

22.4 Code Explanation

```
mapST[a] = b;
```

Character a from s maps to character b from t.

```
mapTS[b] = a;
```

Reverse mapping ensures two different s characters do not map to same t character.

22.5 Dry Run

```
s = "egg"  
t = "add"
```

0	e	a	e -> a
1	g	d	g -> d
2	g	d	consistent

Return true.

22.6 Counterexample

```
s = "ab"  
t = "aa"
```

If only checking s -> t:

```
a -> a  
b -> a
```

This seems consistent individually but invalid because two characters map to same character.

Need reverse map.

22.7 Complexity

Time: $O(n)$

Space: $O(1)$, because 256 ASCII slots.

22.8 Similar Problems

- Word Pattern
- Group Shifted Strings
- Find and Replace Pattern

23. Full Problem Walkthrough 12 — LeetCode 36: Valid Sudoku

23.1 Problem Summary

Determine if a 9x9 Sudoku board is valid.

Rules:

Each digit 1 to 9 must appear at most once in:

- each row
- each column
- each 3x3 box

Empty cells are '.'.

23.2 How to Identify This Pattern

Need to detect duplicates in multiple groups:

- row
- column
- box

Use hash set or boolean arrays.

23.3 C++ Code Using Sets

```
class Solution {
public:
    bool isValidSudoku(vector<vector<char>>& board) {
        vector<unordered_set<char>> rows(9);
        vector<unordered_set<char>> cols(9);
        vector<unordered_set<char>> boxes(9);

        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                char value = board[r][c];

                if (value == '.') {
                    continue;
                }

                int boxIndex = (r / 3) * 3 + (c / 3);

                if (rows[r].count(value) ||
                    cols[c].count(value) ||
                    boxes[boxIndex].count(value)) {
                    return false;
                }

                rows[r].insert(value);
                cols[c].insert(value);
                boxes[boxIndex].insert(value);
            }
        }
    }
};
```

```

    }
}

return true;
}
};

```

23.4 Code Explanation

```
vector<unordered_set<char>> rows(9);
```

Each row has a set of digits seen.

```
int boxIndex = (r / 3) * 3 + (c / 3);
```

Maps cell to one of 9 boxes.

Box indexing:

0-2	0-2	0
0-2	3-5	1
0-2	6-8	2
3-5	0-2	3
3-5	3-5	4
3-5	6-8	5
6-8	0-2	6
6-8	3-5	7
6-8	6-8	8

23.5 Complexity

Board is fixed 9x9.

Time: $O(1)$ technically, because size is fixed.

More generally for $n \times n$ board:

Time: $O(n^2)$

Space: $O(n^2)$

23.6 Edge Cases

- Empty board cells
- Duplicate in row
- Duplicate in column
- Duplicate in box
- Valid partial board

- Board does not need to be solvable, only valid
-

23.7 Common Mistake

Thinking the board must be solvable.

LeetCode 36 only asks whether current filled cells violate Sudoku rules.

23.8 Similar Problems

- Sudoku Solver
 - N-Queens
 - Word Search
-

24. Full Problem Walkthrough 13 — LeetCode 454: 4Sum II

24.1 Problem Summary

Given four integer arrays `nums1`, `nums2`, `nums3`, `nums4`, return number of tuples (i, j, k, l) such that:

$$\text{nums1}[i] + \text{nums2}[j] + \text{nums3}[k] + \text{nums4}[l] == 0$$

24.2 How to Identify This Pattern

Naive 4-loop brute force is $O(n^4)$.

We can split into two pairs:

$a + b$
 $c + d$

Need:

$$a + b = -(c + d)$$

Use hash map to count pair sums.

24.3 Brute Force Idea

```
for a:
  for b:
    for c:
      for d:
        if a+b+c+d == 0
```

Time: $O(n^4)$

24.4 Optimized Intuition

Compute all sums of first two arrays:

`sumAB -> frequency`

Then for every pair in last two arrays, need:

$-(c + d)$

Add its frequency.

24.5 C++ Code

```
class Solution {
public:
    int fourSumCount(vector<int>& nums1, vector<int>& nums2,
                    vector<int>& nums3, vector<int>& nums4) {
        unordered_map<int, int> pairSumCount;

        for (int a : nums1) {
            for (int b : nums2) {
                pairSumCount[a + b]++;
            }
        }

        int answer = 0;

        for (int c : nums3) {
            for (int d : nums4) {
                int need = -(c + d);

                if (pairSumCount.count(need)) {
                    answer += pairSumCount[need];
                }
            }
        }

        return answer;
    }
};
```

24.6 Dry Run

```
nums1 = {1, 2}
nums2 = {-2, -1}
nums3 = {-1, 2}
nums4 = {0, 2}
```

Pair sums $\text{nums1} + \text{nums2}$:

1	-2	-1
1	-1	0
2	-2	0
2	-1	1

Map:

-1 -> 1
0 -> 2
1 -> 1

Now check nums3 + nums4:

-1	0	-1	1	1
-1	2	1	-1	1
2	0	2	-2	0
2	2	4	-4	0

Answer: 2

24.7 Complexity

Let each array length be n.

Time: $O(n^2)$

Space: $O(n^2)$

This is much better than $O(n^4)$.

24.8 Edge Cases

- Duplicate sums
- Negative numbers
- Zeroes
- Large number of tuples

24.9 Common Mistake

Using a set instead of a map.

Need frequencies because multiple pairs can have the same sum.

24.10 Similar Problems

- Two Sum
- 3Sum
- 4Sum
- Count Pairs With Given Sum
- Number of Pairs Satisfying Condition

25. Mini Problem 1 — LeetCode 1207: Unique Number of Occurrences

Pattern

Frequency map + set of counts.

```
class Solution {
public:
    bool uniqueOccurrences(vector<int>& arr) {
        unordered_map<int, int> freq;

        for (int x : arr) {
            freq[x]++;
        }

        unordered_set<int> seenCounts;

        for (auto& p : freq) {
            int count = p.second;

            if (seenCounts.count(count)) {
                return false;
            }

            seenCounts.insert(count);
        }

        return true;
    }
};
```

Key idea:

- 18. Count values
 - 19. Check if counts are unique
-

26. Mini Problem 2 — LeetCode 202: Happy Number

Pattern

Hash set for cycle detection.

```
class Solution {
public:
    int nextNumber(int n) {
        int sum = 0;

        while (n > 0) {
            int digit = n % 10;
            sum += digit * digit;
            n /= 10;
        }

        return sum;
    }

    bool isHappy(int n) {
        unordered_set<int> seen;

        while (n != 1) {
            if (seen.count(n)) {
                return false;
            }

            seen.insert(n);
            n = nextNumber(n);
        }

        return true;
    }
};
```

Key idea:

If a number repeats before reaching 1, we are in a cycle.

27. Mini Problem 3 — LeetCode 290: Word Pattern

Pattern

Two-way mapping.

```
class Solution {
public:
    bool wordPattern(string pattern, string s) {
```

```

vector<string> words;
stringstream ss(s);
string word;

while (ss >> word) {
    words.push_back(word);
}

if (pattern.size() != words.size()) {
    return false;
}

unordered_map<char, string> charToWord;
unordered_map<string, char> wordToChar;

for (int i = 0; i < pattern.size(); i++) {
    char c = pattern[i];
    string w = words[i];

    if (!charToWord.count(c) && !wordToChar.count(w)) {
        charToWord[c] = w;
        wordToChar[w] = c;
    } else {
        if (charToWord[c] != w || wordToChar[w] != c) {
            return false;
        }
    }
}

return true;
};

```

Key idea:

Same as Isomorphic Strings, but mapping char to word.

28. Mini Problem 4 — LeetCode 219: Contains Duplicate II

Pattern

Map value to last index.

```

class Solution {
public:
    bool containsNearbyDuplicate(vector<int>& nums, int k) {
        unordered_map<int, int> lastIndex;

        for (int i = 0; i < nums.size(); i++) {
            int x = nums[i];

            if (lastIndex.count(x)) {
                if (i - lastIndex[x] <= k) {
                    return true;
                }
            }

            lastIndex[x] = i;
        }

        return false;
    }
};

```

Key idea:

For each value, remember most recent index.

29. Mini Problem 5 — LeetCode 409: Longest Palindrome

Pattern

Frequency count.

```

class Solution {
public:
    int longestPalindrome(string s) {
        unordered_map<char, int> freq;

        for (char c : s) {
            freq[c]++;
        }

        int length = 0;
        bool hasOdd = false;

        for (auto& p : freq) {

```

```

    int count = p.second;

    length += (count / 2) * 2;

    if (count % 2 == 1) {
        hasOdd = true;
    }
}

if (hasOdd) {
    length++;
}

return length;
}
};

```

Key idea:

- Use all even counts
 - Use all but one from odd counts
 - One odd character can sit in the center
-

30. Common Edge Cases in Hashing Problems

30.1 Empty Input

Examples:

```

nums = {}
s = ""

```

Expected results:

- Contains Duplicate: false
 - Longest Consecutive: 0
 - First Unique Character: -1
 - Group Anagrams: empty result
-

30.2 One Element

Examples:

```

nums = {5}

```

- Contains Duplicate: false
 - Longest Consecutive: 1
 - Two Sum: no answer unless problem guarantees one
-

30.3 Duplicates

Decide whether duplicates should:

- be ignored: use set
- be counted: use map

Example:

Intersection I uses set.

Intersection II uses map.

30.4 Negative Numbers

Hash maps and sets work fine with negative keys.

Example:

```
unordered_map<int, int> mp;  
mp[-5]++;
```

30.5 Zero

Zero is a normal key.

Important in prefix sum:

```
prefixCount[0] = 1;
```

30.6 Large Sums

Prefix sums can overflow int.

Use:

```
long long prefix = 0;  
unordered_map<long long, int> count;
```

when constraints are large.

30.7 Character Set Assumptions

Use `vector<int> freq(26)` only if input is lowercase English letters.

Otherwise use:

```
unordered_map<char, int>
```

or:

```
vector<int> freq(128)
```

for ASCII.

30.8 Output Order

Unordered containers do not preserve order.

If output order matters, do not rely on iteration order.

30.9 Multiple Valid Answers

Problems like Two Sum may accept any valid answer.

Still ensure indices are valid and distinct.

31. Common Mistakes and Wrong Code

Mistake 1 — Using mp[x] to Check Existence

Wrong:

```
if (mp[x]) {  
    // exists?  
}
```

This inserts x.

Correct:

```
if (mp.count(x)) {  
    // exists  
}
```

Mistake 2 — Using Set When Frequency Is Needed

Wrong for Valid Anagram:

```
unordered_set<char> st;
```

Fails because counts matter.

Use frequency.

Mistake 3 — Forgetting That unordered_map Is Unordered

Wrong assumption:

```
for (auto p : mp) {  
    // first inserted?  
}
```

No ordering guarantee.

Mistake 4 — Forgetting Prefix Initialization

For prefix sum hash map:

```
prefixCount[0] = 1;
```

is crucial.

Mistake 5 — Updating Hash Map in Wrong Order

For Two Sum:

Correct:

```
check complement  
then insert current
```

For Subarray Sum Equals K:

Correct:

update prefix
check need
store prefix

Mistake 6 — Not Handling Duplicates

For intersections, anagrams, pair sums, and frequency problems, duplicates often change the answer.

Mistake 7 — Assuming Average $O(1)$ Means Guaranteed $O(1)$

Technically, unordered containers have worst-case $O(n)$, though interviews usually accept average $O(1)$.

32. Pattern Recognition Checklist

Use this during interviews.

Ask:

20. Do I need fast lookup?
21. Do I need to know if something appeared before?
22. Do I need to count frequencies?
23. Do I need to return indices?
24. Do I need to group items?
25. Do duplicates matter?
26. Is order irrelevant?
27. Do I need prefix sum counts?
28. Would brute force repeatedly search?
29. Is $O(n)$ extra space acceptable?

If yes, use hashing.

33. Interview Explanation Templates

33.1 For Hash Set Duplicate Problems

I use a hash set to store values I have already seen. As I scan the array, if the current value is already in the set, I know a duplicate exists. Otherwise, I insert it. This gives average $O(n)$ time and $O(n)$ space.

33.2 For Two Sum

For each number, I compute the complement needed to reach the target. If that complement has already appeared, I return its index and the current index. Otherwise, I store the current number and index for future lookup.

33.3 For Frequency Counting

I count how many times each value appears using a hash map or frequency array. Then I use those counts to check the required condition, such as anagram equality, uniqueness, or availability.

33.4 For Grouping

I convert each item into a canonical key. Items with the same key belong to the same group. I use a hash map from key to list of items.

33.5 For Prefix Sum + Hash Map

I maintain the running prefix sum. A subarray ending at the current index has sum k if there was a previous prefix sum equal to currentPrefix - k. So I store counts of previous prefix sums in a hash map.

34. Complexity Patterns

Duplicate detection	$O(n)$ average	$O(n)$
Frequency count	$O(n)$ average	$O(k)$
Two Sum	$O(n)$ average	$O(n)$
Valid Anagram with 26 array	$O(n)$	$O(1)$
Group Anagrams with sorting key	$O(n * m \log m)$	$O(n * m)$
Group Anagrams with frequency key	$O(n * m)$	$O(n * m)$
Longest Consecutive Sequence	$O(n)$ average	$O(n)$
Subarray Sum Equals K	$O(n)$ average	$O(n)$
4Sum II	$O(n^2)$ average	$O(n^2)$
Valid Sudoku	$O(1)$ for 9x9	$O(1)$

Here:

- n = number of elements
 - m = average string length
 - k = number of unique keys
-

35. Time-Boxed Practice Guidance

Easy Hash Set / Frequency Problems

Target time:

5–12 minutes

Examples:

- Contains Duplicate

- Valid Anagram
 - Ransom Note
 - First Unique Character
-

Core Medium Hashing Problems

Target time:

15–25 minutes

Examples:

- Two Sum
 - Group Anagrams
 - Longest Consecutive Sequence
 - Subarray Sum Equals K
-

Harder Hashing / Prefix Hash Problems

Target time:

25–40 minutes

Examples:

- 4Sum II
 - Valid Sudoku
 - Subarray Sums Divisible by K
 - Contiguous Array
-

36. Expanded Practice Set

36.1 Must-Do Easy

- 1.1. Contains Duplicate
 - 2.2. Valid Anagram
 - 3.3. Ransom Note
 - 4.4. First Unique Character in a String
 - 5.5. Intersection of Two Arrays
 - 6.6. Intersection of Two Arrays II
 - 7.7. Unique Number of Occurrences
 - 8.8. Happy Number
 - 9.9. Word Pattern
 - 10.10. Longest Palindrome
 - 11.11. Contains Duplicate II
 - 12.12. Isomorphic Strings
-

36.2 Must-Do Medium

- 13.13. Two Sum
- 14.14. Group Anagrams
- 15.15. Longest Consecutive Sequence
- 16.16. Subarray Sum Equals K
- 17.17. Valid Sudoku
- 18.18. 4Sum II
- 19.19. Contiguous Array

20.20.	Subarray Sums Divisible by K
21.21.	Continuous Subarray Sum
22.22.	Binary Subarrays With Sum
23.23.	Count Number of Nice Subarrays
24.24.	K-diff Pairs in an Array

36.3 Strong Interview Practice

25.25.	Top K Frequent Elements
26.26.	Sort Characters By Frequency
27.27.	Insert Delete GetRandom O(1)
28.28.	LRU Cache
29.29.	Time Based Key-Value Store
30.30.	Find Duplicate File in System
31.31.	Brick Wall
32.32.	4Sum, often sorting/two pointers
33.33.	Substring with Concatenation of All Words
34.34.	Minimum Window Substring, sliding window + hashing

36.4 Hard / Stretch

35.35.	Palindrome Pairs
36.36.	Max Points on a Line
37.37.	All O(1) Data Structure
38.38.	LFU Cache
39.39.	Number of Atoms, parsing + map
40.40.	Maximum Frequency Stack

37. Suggested Study Order

Step 1 — Basic Existence

Start with:

- 30. Contains Duplicate
- 31. Intersection of Two Arrays
- 32. Happy Number

Goal:

- Learn `unordered_set`
 - Practice insert and count
-

Step 2 — Frequency Counting

Then solve:

- 33. Valid Anagram
- 34. Ransom Note
- 35. First Unique Character
- 36. Unique Number of Occurrences

Goal:

- Learn frequency map/array
 - Understand when counts matter
-

Step 3 — Value to Index

Solve:

- 37. Two Sum
- 38. Contains Duplicate II
- 39. Isomorphic Strings
- 40. Word Pattern

Goal:

- Learn mapping values to positions or relationships
-

Step 4 — Grouping

Solve:

- 41. Group Anagrams
- 42. Group Shifted Strings
- 43. Find Duplicate File in System

Goal:

- Learn canonical keys/signatures
-

Step 5 — Advanced Hashing

Solve:

- 44. Longest Consecutive Sequence
- 45. Subarray Sum Equals K
- 46. Contiguous Array
- 47. Subarray Sums Divisible by K
- 48. 4Sum II

Goal:

- Learn hash set expansion
 - Learn prefix sum + hash map
 - Learn pair-sum counting
-

38. Pattern Decision Table

“contains duplicate”	unordered_set	Contains Duplicate
“seen before”	unordered_set	Happy Number
“return indices”	unordered_map<value,index>	Two Sum
“frequency/count”	unordered_map<T,int>	Valid Anagram
“lowercase letters”	vector<int>(26)	Ransom Note
“group anagrams”	map key to vector	Group Anagrams
“unique result”	unordered_set	Intersection I

Problem Clue	Use	Example
“duplicates matter”	frequency map	Intersection II
“subarray sum equals k”	prefix count map	Subarray Sum Equals K
“longest consecutive”	hash set start expansion	Longest Consecutive
“one-to-one mapping”	two maps	Isomorphic Strings
“row/col/box duplicate”	sets	Valid Sudoku
“count tuples from four arrays”	pair sum map	4Sum II

39. Final Cheat Sheet

Hash Set

```
unordered_set<int> seen;
```

```
for (int x : nums) {
    if (seen.count(x)) {
        return true;
    }

    seen.insert(x);
}
```

Frequency Map

```
unordered_map<int, int> freq;
```

```
for (int x : nums) {
    freq[x]++;
}
```

Character Frequency

```
vector<int> freq(26, 0);
```

```
for (char c : s) {
```

```
    freq[c - 'a']++;  
}
```

Two Sum

```
unordered_map<int, int> index;  
  
for (int i = 0; i < nums.size(); i++) {  
    int need = target - nums[i];  
  
    if (index.count(need)) {  
        return {index[need], i};  
    }  
  
    index[nums[i]] = i;  
}
```

Group Anagrams

```
unordered_map<string, vector<string>> groups;  
  
for (string word : strs) {  
    string key = word;  
    sort(key.begin(), key.end());  
  
    groups[key].push_back(word);  
}
```

Prefix Sum Count

```
unordered_map<int, int> count;  
count[0] = 1;  
  
int prefix = 0;  
int answer = 0;  
  
for (int x : nums) {  
    prefix += x;  
  
    if (count.count(prefix - k)) {  
        answer += count[prefix - k];  
    }  
}
```

```
count[prefix]++;  
}
```

Longest Consecutive

```
unordered_set<int> st(nums.begin(), nums.end());  
  
int best = 0;  
  
for (int x : st) {  
    if (!st.count(x - 1)) {  
        int current = x;  
        int length = 1;  
  
        while (st.count(current + 1)) {  
            current++;  
            length++;  
        }  
  
        best = max(best, length);  
    }  
}
```

Two-Way Mapping

```
unordered_map<char, char> aToB;  
unordered_map<char, char> bToA;
```

Use when mapping must be one-to-one.

40. What to Master Before Chapter 3

Before moving to Two Pointers, make sure you can:

- Use unordered_set for existence checks
- Use unordered_map for frequencies
- Use hash map for value-to-index mapping
- Solve Two Sum
- Solve Valid Anagram
- Solve Group Anagrams
- Solve Longest Consecutive Sequence
- Understand prefix sum + hash map
- Know why prefixCount[0] = 1 matters
- Know why mp[x] can insert a key

- Know when to use vector frequency instead of map
 - Know when duplicates require a map instead of set
 - Avoid relying on unordered iteration order
 - Explain average $O(1)$ lookup
 - Explain $O(n)$ space tradeoff
-

41. Core Takeaway

Hashing is about remembering information so you can access it quickly later.

The core question is:

What do I need to look up quickly?

If the answer is:

- whether a value exists
- how many times a value appeared
- where a value appeared
- which group a value belongs to
- how many times a prefix sum appeared

then hashing is probably the right pattern.

Most hashing solutions follow this structure:

create hash map or hash set

for each element:

check something in hash table

update answer

update hash table

return answer

Hashing is one of the biggest interview foundations because it appears inside many later patterns:

- sliding window
- prefix sum
- graph visited sets
- trie alternatives
- design data structures
- heap + frequency problems
- union/grouping problems

Master this chapter well before moving forward.
